

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 08 | Singly Linked List: Nodes, Pointers and Dynamic Memory

Week 8 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 08 at a glance

Item	Details
Lab title	Singly Linked List: Nodes, Pointers and Dynamic Memory
Week	8 of 16
Prerequisites	Lab 01 (references in memory), Lab 03 (arrays, classes)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab08_Instructor.ipynb , Lab08_Student.ipynb
Next lab	Lab 09: Doubly and Circular Linked Lists

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain how a linked list stores data in **nodes connected by references (pointers)**.
2. Compare linked lists with arrays in terms of memory and operation costs.
3. Implement a **Node** class and a **LinkedList** class with **head**, **tail** and **size**.
4. Implement **traversal**, **search**, **insertion** (head, tail, position) and **deletion** (head, value, position).
5. **Reverse** a linked list in place by re-linking pointers.
6. Use the **slow/fast pointer** technique to find the middle and to detect a cycle.
7. Solve classic interview problems on linked lists.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-1	Explain the structure of linked lists	Understand (C2)
CLO-2	Implement singly linked list operations	Apply (C3)
CLO-3	Compare arrays and linked lists	Analyse (C4)

2. Motivation: When Arrays Are Not Enough

Arrays are fast for access, but they have two big problems: their size is fixed (or resizing needs copying), and **inserting or deleting in the middle requires shifting** many elements. A **linked list** solves this by not storing elements side by side at all. Each element simply **remembers where the next one is**.



Analogy: a train

A train is a chain of coaches. Each coach is connected to the next one. To add a coach in the middle, you **unhook** two coaches and hook the new one in between; no other coach has to move. To reach coach 7, however, you must walk through coaches 1 to 6. That is a linked list.

2.1 Linked Lists in Daily Life

Example	Why it is like a linked list
Treasure hunt / clue chain	Each clue tells you where the next clue is

Example	Why it is like a linked list
Music playlist (next song)	Each song points to the next song
Image viewer (next image)	Each image knows the next image
Round-robin turns in a Ludo game	Each player knows who plays next
A queue of people where everyone only knows who is behind them	Follow the chain from the first person



2.2 Linked Lists in Industry

- **Operating systems** keep lists of processes, free memory blocks and open files as linked lists.
- **Undo/redo, browser history, and music players** use linked lists (often doubly linked, Lab 09).
- **Hash tables** (Lab 12) use linked lists to handle collisions (separate chaining).
- **Stacks, queues and graphs** (adjacency lists) are commonly built from linked lists.
- **Blockchain** is essentially a linked list where each block stores the hash (reference) of the previous block.



2.3 Linked Lists in Research

- **Memory management and garbage collection** research (e.g., in Python and Java runtimes) is built around linked structures.
- **Lock-free linked lists** are studied for high-performance multi-core and database systems.
- **Sparse data:** polynomials and sparse matrices with mostly zeros are stored efficiently as linked lists of non-zero terms.



2.4 Why It Matters in 2026–2030



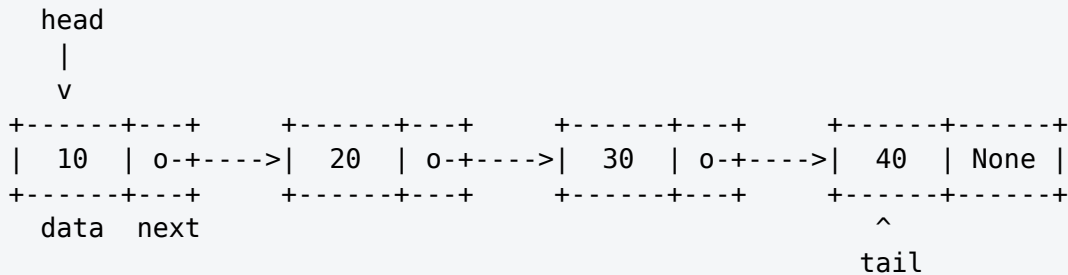
Pointers are the language of complex data structures

Trees, graphs, tries and many AI data structures (e.g., knowledge graphs, computation graphs in PyTorch) are just **nodes with references**. If you can confidently create, re-link and traverse nodes in a linked list, you have mastered the core skill for everything that comes next in this course and in advanced systems.

3. Part A: Structure of a Linked List

A **singly linked list** is a sequence of **nodes**. Each node has two parts: the **data** and a reference called **next** that points to the following node. The first node is the **head**; the last node's **next** is **None** (NULL).

A singly linked list



The nodes can be anywhere in memory, e.g. addresses 5000, 1200, 8800, 3400

	Array	Linked List
Memory	Contiguous block	Nodes scattered anywhere
Size	Fixed or resized by copying	Grows one node at a time
Access k-th element	$O(1)$	$O(n)$, must walk from head
Insert / delete at front	$O(n)$, shifting	$O(1)$
Insert / delete at end	$O(1)$ amortised	$O(1)$ with a tail pointer (insert)
Insert after a known node	$O(n)$	$O(1)$
Extra memory	None per element	One reference per node
Cache performance	Excellent	Poorer (nodes are far apart)



4. Part B: The Node and LinkedList Classes

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None          # reference to the next node

# Building a list by hand: 10 -> 20 -> 30
a = Node(10); b = Node(20); c = Node(30)
a.next = b
b.next = c
head = a

current = head                    # traversal
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
print("None")

```

Now we wrap this in a **LinkedList** class that keeps track of **head**, **tail** and **size**, so users of our list never handle nodes directly.

```

class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):
        return self.size

    def __str__(self):
        parts, cur = [], self.head
        while cur:
            parts.append(str(cur.data))
            cur = cur.next
        return " -> ".join(parts + ["None"])

    def is_empty(self):
        return self.head is None

```

+ 5. Part C: Insertion

5.1 Insert at the Head: $O(1)$

Steps (order matters!)

Before: head -> [20] -> [30] -> None

1. new = Node(10)

2. new.next = head [10] -> [20] -> [30] -> None

3. head = new head -> [10] -> [20] -> [30] -> None

5.2 Insert at the Tail: $O(1)$ with a tail pointer

Without a tail pointer we would have to walk the whole list to find the last node ($O(n)$). With **tail**, we simply do **tail.next = node** and then **tail = node**. If the list is empty, the new node is both head and tail.

5.3 Insert at a Position: $O(n)$

Walk to the node **before** the position (index - 1), then link the new node in between. Always connect the new node to the rest of the list **before** changing the previous node's **next**, or you will lose the rest of the list.

```

def push_front(self, data):
    node = Node(data)
    node.next = self.head
    self.head = node
    if self.tail is None:          # list was empty
        self.tail = node
    self.size += 1

def push_back(self, data):
    node = Node(data)
    if self.head is None:
        self.head = self.tail = node
    else:
        self.tail.next = node
        self.tail = node
    self.size += 1

def insert_at(self, index, data):
    if index < 0 or index > self.size:
        raise IndexError("index out of range")
    if index == 0:
        return self.push_front(data)
    if index == self.size:
        return self.push_back(data)
    prev = self.head
    for _ in range(index - 1):      # walk to the node before index
        prev = prev.next
    node = Node(data)
    node.next = prev.next          # 1. connect new node to the rest
    prev.next = node              # 2. connect previous node to new node
    self.size += 1

LinkedList.push_front = push_front
LinkedList.push_back = push_back
LinkedList.insert_at = insert_at

ll = LinkedList()
for x in [20, 30, 50]:
    ll.push_back(x)
ll.push_front(10)
ll.insert_at(3, 40)
print(ll, "| size =", len(ll))

```

We attach methods to the class one part at a time so each idea stays short. In your own code, write all methods inside the class.

6. Part D: Search and Access

```
def find(self, target):
    cur, index = self.head, 0
    while cur:
        if cur.data == target:
            return index
        cur = cur.next
        index += 1
    return -1

def get(self, index):
    if not 0 <= index < self.size:
        raise IndexError("index out of range")
    cur = self.head
    for _ in range(index):
        cur = cur.next
    return cur.data

LinkedList.find = find
LinkedList.get = get
print(ll.find(40), ll.find(99), ll.get(2))
```

— 7. Part E: Deletion

To delete a node, make the **previous node skip over it**: `prev.next = prev.next.next`. Python's garbage collector frees the removed node automatically (in C++ you must **delete** it yourself). Special cases: deleting the head, deleting the tail (update **tail**), and an empty list.

Skipping a node

```
Delete 30:   head -> [10] -> [20] -> [30] -> [40] -> None
              prev   target
prev.next = target.next
              head -> [10] -> [20] -----> [40] -> None
```

```
def pop_front(self):
    if self.head is None:
        raise IndexError("pop from empty list")
    data = self.head.data
    self.head = self.head.next
    if self.head is None:
        self.tail = None
    self.size -= 1
    return data

def remove(self, target):
    """Remove the first node containing target. Return True if removed."""
    if self.head is None:
        return False
    if self.head.data == target:
        self.pop_front()
        return True
    prev = self.head
    while prev.next and prev.next.data != target:
        prev = prev.next
    if prev.next is None:
        return False # not found
    if prev.next is self.tail:
        self.tail = prev # deleting the last node
    prev.next = prev.next.next
    self.size -= 1
    return True

LinkedList.pop_front = pop_front
LinkedList.remove = remove
print(ll)
ll.remove(30); ll.remove(50); print(ll, "| tail =", ll.tail.data)
print("popped", ll.pop_front(), "->", ll)
```

8. Part F: Reversing a Linked List

Reversing is the most famous linked-list interview question. We walk through the list once and **flip each next pointer** to point backwards. We need three references: **prev**, **cur** and **nxt**.

Dry run

```
Start:   None   [10] -> [20] -> [30] -> None
         prev   cur
Step 1:  None <- [10]   [20] -> [30] -> None
         prev   cur
Step 2:  None <- [10] <- [20]   [30] -> None
         prev   cur
Step 3:  None <- [10] <- [20] <- [30]         head = prev
```

```
def reverse(self):
    prev, cur = None, self.head
    self.tail = self.head
    while cur:
        nxt = cur.next           # 1. save the rest of the list
        cur.next = prev         # 2. flip the pointer
        prev = cur              # 3. move prev forward
        cur = nxt               # 4. move cur forward
    self.head = prev

LinkedList.reverse = reverse
demo = LinkedList()
for x in [10, 20, 30, 40]:
    demo.push_back(x)
demo.reverse()
print(demo)
```

Time $O(n)$, extra space $O(1)$. Notice how step 1 is essential: if you flip the pointer first, you lose the rest of the list forever.

9. Part G: Slow and Fast Pointers

9.1 Find the Middle in One Pass

Move **slow** one step and **fast** two steps at a time. When **fast** reaches the end, **slow** is at the middle.

```
def middle(self):
    slow = fast = self.head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow.data if slow else None

LinkedList.middle = middle
m = LinkedList()
for x in [1, 2, 3, 4, 5]:
    m.push_back(x)
print("middle of", m, "is", m.middle())
```

9.2 Detect a Cycle (Floyd's Tortoise and Hare)

If some node's **next** points back to an earlier node, traversal would loop forever. With slow and fast pointers, if there is a cycle the fast pointer will eventually **meet** the slow one, like a faster runner lapping a slower runner on a circular track.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

nodes = [Node(i) for i in range(5)]
for i in range(4):
    nodes[i].next = nodes[i + 1]
print("cycle?", has_cycle(nodes[0]))
nodes[4].next = nodes[2]          # create a cycle 4 -> 2
print("cycle?", has_cycle(nodes[0]))
```

10. C++ Corner

In C++, nodes are created with **new** and must be freed with **delete**. Forgetting to free them causes **memory leaks**.

C++

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};

class LinkedList {
    Node* head = nullptr;
public:
    void pushFront(int d) { Node* n = new Node(d); n->next = head; head = n; }
    bool remove(int target) {
        Node** cur = &head;           // pointer to the link we may
change
        while (*cur && (*cur)->data != target) cur = &((*cur)->next);
        if (!*cur) return false;
        Node* dead = *cur;
        *cur = dead->next;
        delete dead;                   // free memory
        return true;
    }
    void reverse() {
        Node *prev = nullptr, *cur = head;
        while (cur) { Node* nxt = cur->next; cur->next = prev; prev = cur; cur
= nxt; }
        head = prev;
    }
    void print() {
        for (Node* c = head; c; c = c->next) cout << c->data << " -> ";
        cout << "NULL" << endl;
    }
    ~LinkedList() { while (head) { Node* t = head; head = head->next; delete
t; } }
};

int main() {
    LinkedList ll;
    for (int x : {40, 30, 20, 10}) ll.pushFront(x);
    ll.print(); ll.remove(30); ll.reverse(); ll.print();
}

```

11. Complexity Summary

Operation	Singly linked list (head + tail)
push_front / pop_front	O(1)
push_back	O(1)
pop_back (remove last)	O(n), need the node before tail

Operation	Singly linked list (head + tail)
get(i) / find(x)	O(n)
insert_at / remove(x)	O(n) to find the spot, O(1) to re-link
reverse / middle / has_cycle	O(n) time, O(1) space

🚫 12. Common Mistakes

Mistake	Effect	Fix
Moving head while traversing	Lose the list	Use a separate cur variable
Changing prev.next before linking the new node	Rest of the list is lost	Link node.next first
Not handling an empty list	AttributeError: 'NoneType'...	Check if self.head is None
Forgetting to update tail	push_back attaches to a deleted node	Update tail when deleting the last node
while cur.next: when cur may be None	Crash on empty list	Check cur first or use while cur:
Forgetting size updates	Wrong length	Change size in every insert/delete

13. Lab Tasks

Solve all tasks in `Lab08_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Build and Print

The notebook gives helper functions `from_list(values)` (builds a list and returns its head) and `to_list(head)`.

Write `length(head)` and `sum_values(head)` iteratively, and `length_rec(head)` recursively.

```
head = from_list([4, 8, 15]) → length 3, sum 27
```

Task 2 Remove All Occurrences

Write `remove_all(head, value)` that deletes **every** node containing `value` and returns the (possibly new) head. Hint: a **dummy node** placed before the head removes the special case of deleting the head.

```
[7, 1, 7, 2, 7] remove 7 → [1, 2]
```

Task 3 Reverse (Iterative and Recursive)

Write `reverse_iter(head)` and `reverse_rec(head)`, each returning the new head. For the recursive version: reverse the rest, then make the second node point back to the first.

```
[1, 2, 3, 4] → [4, 3, 2, 1]
```

Task 4 Merge Two Sorted Lists

Two sorted linked lists of roll numbers must be merged into one sorted list by **re-linking the existing nodes** (do not create new nodes except one dummy). Return the head.

```
[1, 4, 9] + [2, 3, 10] → [1, 2, 3, 4, 9, 10]
```

Task 5 Remove the N-th Node from the End

In **one pass**, remove the n-th node from the end and return the head. Hint: move a **fast** pointer n steps ahead, then move **fast** and **slow** together; use a dummy node.

[1, 2, 3, 4, 5], n = 2 → [1, 2, 3, 5]

Task 6 Palindrome Linked List

Check whether a linked list reads the same forwards and backwards in **O(n) time and O(1) extra space**: find the middle (slow/fast), reverse the second half, compare the two halves, and (good practice) restore the list.

[1, 2, 2, 1] → True, [1, 2, 3] → False

Task 7 Add Two Big Numbers

Very large numbers (bigger than any built-in integer type in C++) can be stored as linked lists of digits in **reverse order**. Add two such numbers and return the result as a linked list.

This is how arbitrary-precision arithmetic (used in cryptography and Python's own big integers) works.

342 + 465: [2, 4, 3] + [5, 6, 4] → [7, 0, 8] (= 807)



14. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab08** folder.

#	Problem	Level	Concept practised
206	Reverse Linked List	Easy	Pointer re-linking (Task 3)
21	Merge Two Sorted Lists	Easy	Dummy node (Task 4)
876	Middle of the Linked List	Easy	Slow / fast pointers
203	Remove Linked List Elements	Easy	Dummy node (Task 2)
141	Linked List Cycle	Easy	Floyd's algorithm
234	Palindrome Linked List	Easy	Middle + reverse (Task 6)
19	Remove Nth Node From End of List	Medium	Two pointers (Task 5)

#	Problem	Level	Concept practised
2	Add Two Numbers	Medium	Digit lists (Task 7)



15. Home Task

1. Finish all tasks and push `Lab08_Student.ipynb` to GitHub.
2. Add `pop_back()`, `insert_sorted(x)` and `__iter__` (so `for x in ll:` works) to your `LinkedList` class.
3. Find the **start node** of a cycle using Floyd's algorithm (after the meeting point, move one pointer to head and advance both by one).
4. **Mini-project: Polynomial Calculator.** Store polynomials such as $5x^3 + 4x^2 + 2$ as linked lists of (coefficient, power) nodes sorted by power. Implement addition, multiplication and evaluation at a given x .
5. Solve at least **3** LeetCode problems from the list.



16. Viva Questions

1. What is a node? What are its two parts?
2. What does the head point to? What does the last node's next contain?
3. Why is accessing the k -th element $O(n)$ in a linked list?
4. Why is inserting at the front $O(1)$ in a linked list but $O(n)$ in an array?
5. Why do we keep a tail pointer?
6. Explain the order of steps when inserting a node in the middle.
7. How do you delete a node? What special cases must you handle?
8. Explain how to reverse a linked list in place.
9. How does the slow/fast pointer technique find the middle?
10. How does Floyd's algorithm detect a cycle?



17. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain

Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.

18. References and Further Learning

- Linked list animations: **visualgo.net/en/list**
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.